

Programowanie Obiektowe – Projekt Indywidualny

VaultCore – System Bankowy

Autor

Kamil Kalicki

Przedmiot

Programowanie Obiektowe

Rok akademicki

2025/2026

1. Opis zadania

Celem projektu było zaprojektowanie oraz implementacja aplikacji bankowej w języku Java zgodnie z zasadami programowania obiektowego. Program umożliwia zarządzanie użytkownikami oraz kontami bankowymi, wykonywanie podstawowych operacji finansowych, zapisywanie historii transakcji oraz trwałe przechowywanie danych.

Projekt został wykonany z wykorzystaniem biblioteki Swing do stworzenia graficznego interfejsu użytkownika.

Założenia projektu

- obsługa klientów banku,
- obsługa różnych typów kont,
- wykonywanie operacji finansowych,
- historia transakcji,
- trwałość danych,
- logowanie do aplikacji,
- wykorzystanie zasad OOP.

Ograniczenia

- brak bazy danych,
- uproszczony system logowania,
- uproszczona walidacja adresów e-mail.

2. Diagram klas UML

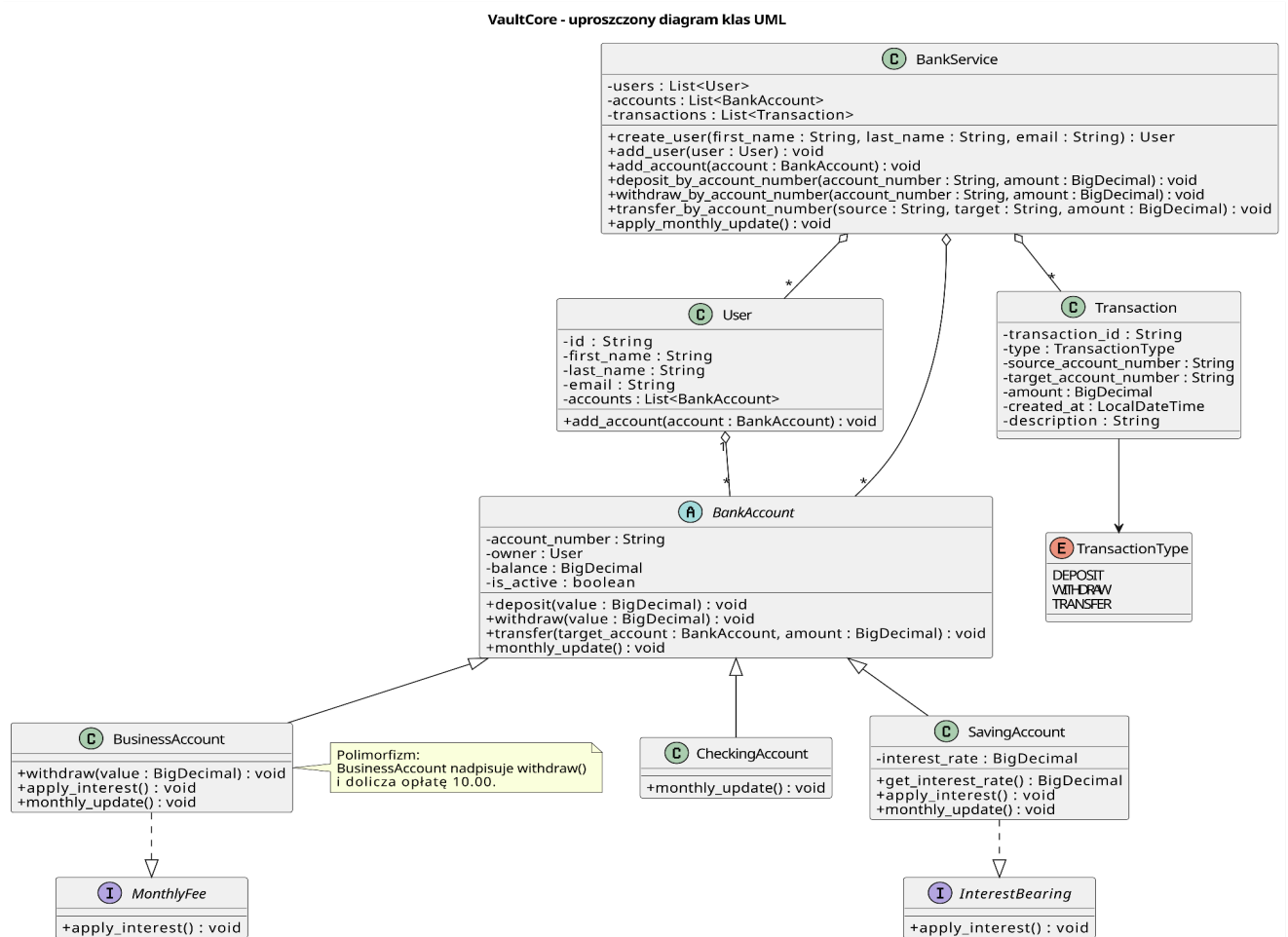


Diagram przedstawia strukturę aplikacji, zależności pomiędzy klasami, zastosowane dziedziczenie, implementację interfejsów oraz relacje pomiędzy komponentami systemu.

3. Instrukcja użytkownika

Uruchomienie aplikacji

1. Uruchomić klasę Main.
2. Podać login i hasło.
3. Po poprawnym zalogowaniu zostanie wyświetlone główne okno aplikacji.

Dodawanie klienta

1. Przejść do zakładki Clients.
2. Wprowadzić dane klienta.
3. Kliknąć przycisk Add User.

Tworzenie konta

1. Przejść do zakładki Accounts.
2. Wybrać użytkownika.
3. Wybrać typ konta.
4. W przypadku konta oszczędnościowego podać oprocentowanie.
5. Kliknąć Create Account.

Operacje finansowe

Aplikacja umożliwia:

- wpłatę środków,
- wypłatę środków,
- przelew pomiędzy kontami,
- wykonanie miesięcznej aktualizacji.

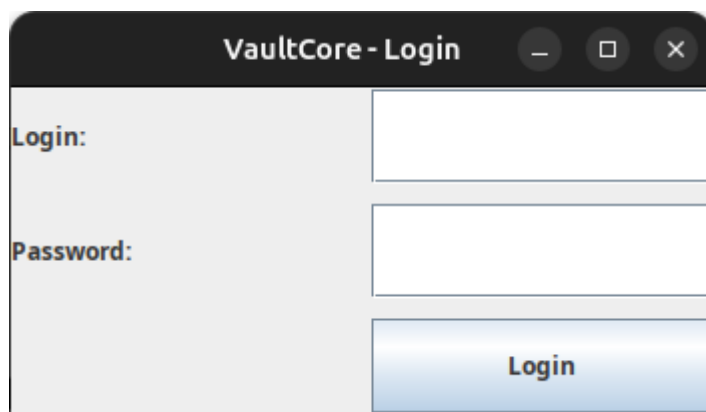
Historia transakcji

W zakładce Transactions wyświetlana jest historia wszystkich wykonanych operacji.

4. Najważniejsze funkcjonalności programu

4.1 Logowanie do aplikacji

Po uruchomieniu programu użytkownik musi podać login oraz hasło. Dopiero po poprawnej autoryzacji możliwe jest korzystanie z systemu.



The image shows a window titled "VaultCore - Login". It contains two text input fields: the first is labeled "Login:" and the second is labeled "Password:". Below these fields is a blue button with the text "Login". The window has a standard macOS-style title bar with minimize, maximize, and close buttons.

4.2 Zarządzanie klientami

System umożliwia tworzenie klientów banku wraz z podstawowymi danymi identyfikacyjnymi.

VaultCore

Clients Accounts Operations Transactions

First name: Last name: Email: Add user

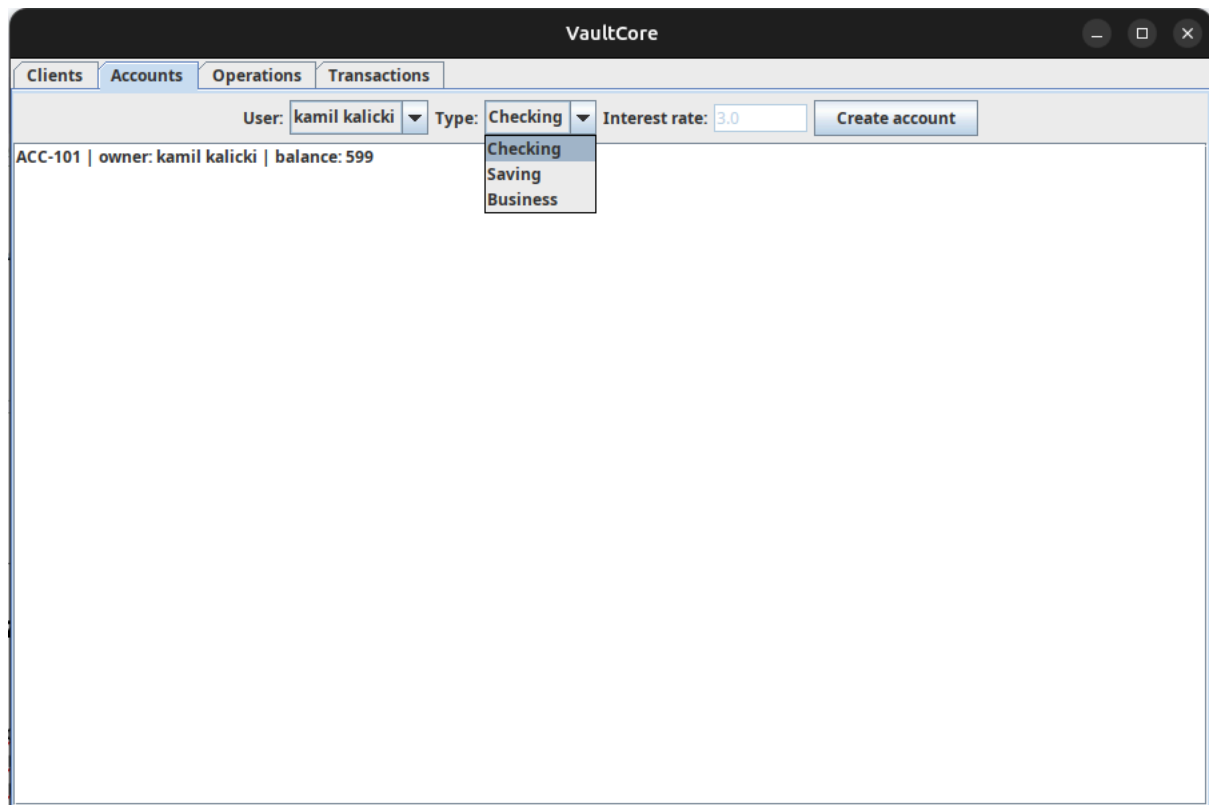
kamil kalicki | Kamil@gmail.com

4.3 Zarządzanie kontami

Użytkownik może tworzyć trzy rodzaje kont:

- CheckingAccount,
- SavingAccount,
- BusinessAccount.

Każdy klient może posiadać wiele kont bankowych.

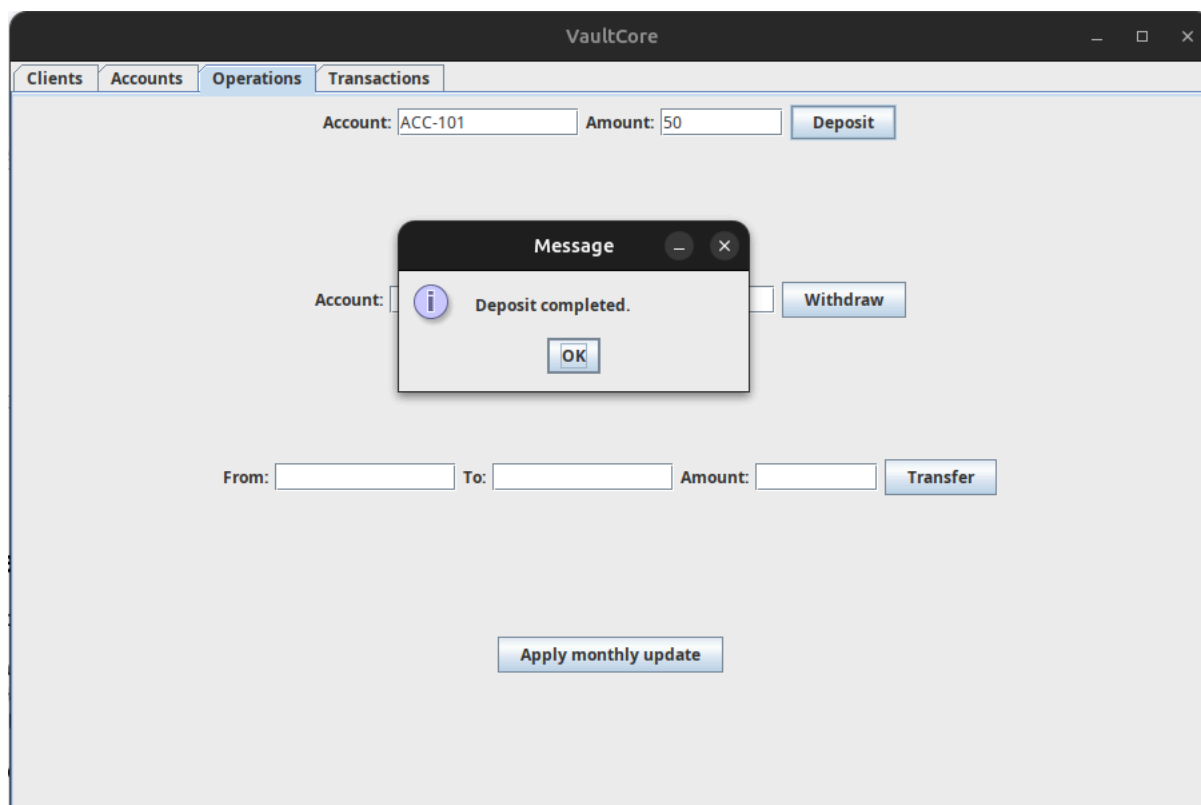


4.4 Operacje finansowe

System umożliwia realizację podstawowych operacji finansowych:

- wpłaty,
- wypłaty,
- przelewy.

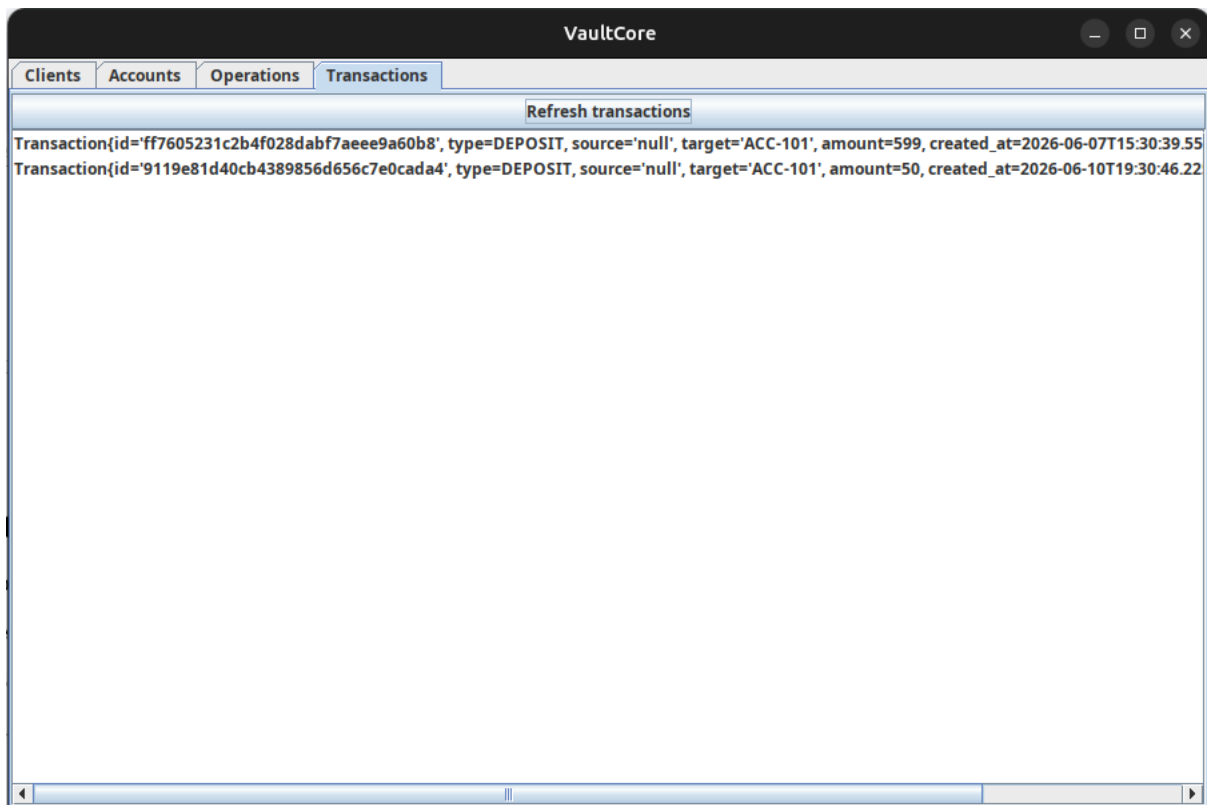
Wszystkie operacje są zapisywane w historii transakcji.



4.5 Historia transakcji

Każda wykonana operacja jest zapisywana w postaci obiektu Transaction.

Historia operacji dostępna jest w zakładce Transactions.



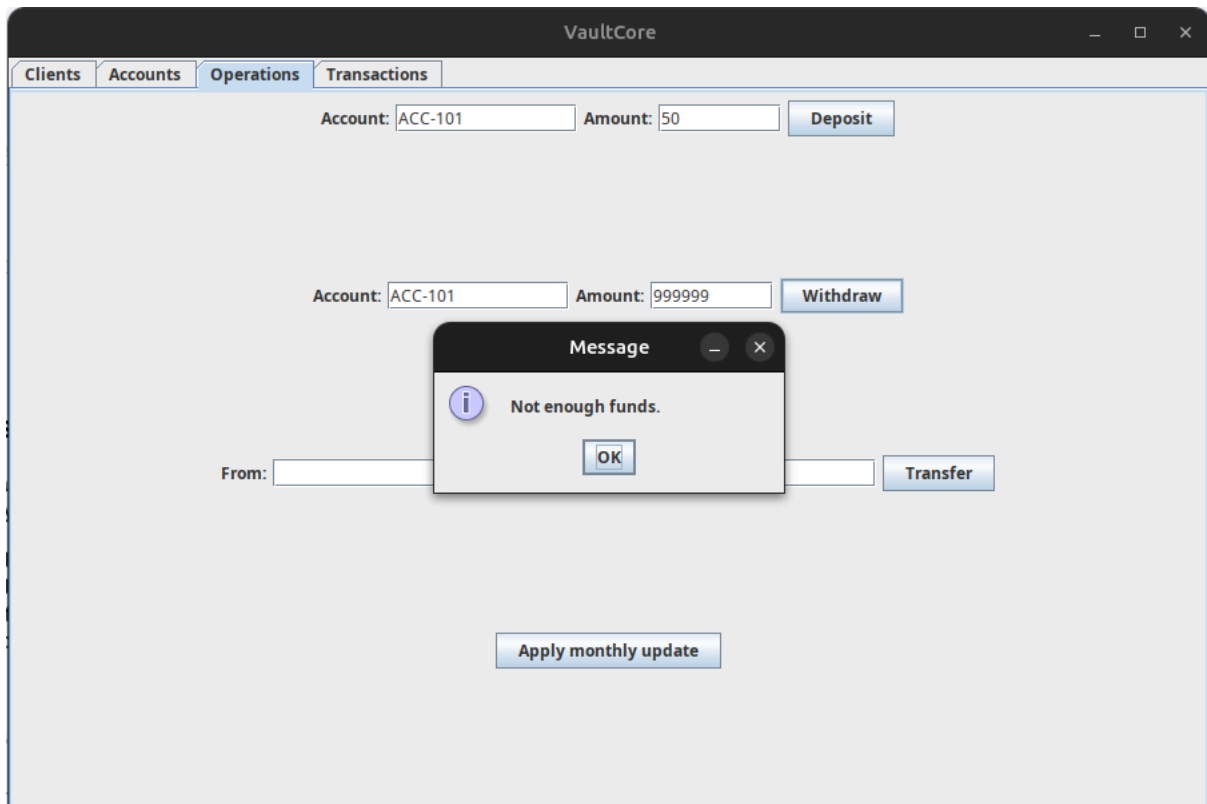
4.6 Obsługa wyjątków

Aplikacja zabezpiecza się przed niepoprawnymi operacjami poprzez wykorzystanie własnych wyjątków:

- `InvalidAmountException`,
- `InvalidEmailException`,

- InsufficientFundsException,
- AccountNotFoundException.

Przykładem jest próba wypłaty większej kwoty niż dostępne saldo.



4.7 Dziedziczenie i polimorfizm

Klasa BankAccount została zaimplementowana jako klasa abstrakcyjna.

Dziedziczą po niej:

- CheckingAccount,
- SavingAccount,

- BusinessAccount.

Przykładem polimorfizmu jest nadpisanie metody `withdraw()` w klasie `BusinessAccount`. Każda wypłata z konta firmowego powoduje automatyczne doliczenie opłaty 10.00.

Drugim przykładem jest metoda `monthly_update()`, która działa inaczej dla każdego typu konta.

5. Podsumowanie

Projekt VaultCore spełnia założenia postawione w zadaniu. W aplikacji wykorzystano najważniejsze elementy programowania obiektowego:

- enkapsulację,
- dziedziczenie,
- polimorfizm,
- klasy abstrakcyjne,
- interfejsy,
- obsługę wyjątków.

Program posiada graficzny interfejs użytkownika oraz umożliwia trwałe przechowywanie danych przy użyciu serializacji.